
Objetivo do Material

Sumário

Objetivo do Material	3
Como usar este playbook	4
Mapa dos Estágios	5
Passos Práticos	6
Passo 1 — O Fim do Digitador de Faturas	6
Passo 2 — Entendendo a Dinâmica de Compras do Dentista Português	9
Passo 3 — Criando a Ponte Comercial-Financeiro	14
Passo 4 — O Seu Plano de Ação Imediato	18
Checklist Mestre	23

Objetivo do Material

Apresentar a transformação do analista operacional para o estratégico — o gancho emocional de quem se sente preso em tarefas repetitivas e quer acessar o próximo nível da carreira.

Como usar este playbook

Você é o **Analista Estratégico**. Cada passo é um card independente com sete partes: objetivo, pré-requisito, entregas, execução, gate de verificação, critério de conclusão e armadilhas.

Este documento **não repete a teoria** do livro. Quando precisar do porquê, siga a referência cruzada do card para o capítulo correspondente.

Mapa dos Estágios

#	Estágio	Passos
1	Passe	1, 2
2	Acesso	3, 4

Passos Práticos

Passo 1 — O Fim do Digitador de Faturas

Estágio: Passe · **Origem:** Cap. 1 — O Fim do Digitador de Faturas

① Objetivo do passo

Fazer o leitor mapear sua semana de trabalho e identificar gargalos operacionais que serão substituídos por IA ao longo da série.

② Pré-requisito

Nenhum — este é o ponto de partida

③ Entregas

- (a completar)

④ Execução

Passo 1: Planejar o Mapa de Processos

```
# mapear_processos.py
# Framework PASSO - Passo 1: Mapeamento de Processos Financeiros
# Este script cria um inventário completo dos processos financeiros
# de um fornecedor de odontologia, classificando cada um por nível
# de automatização e impacto no fluxo de caixa.

import pandas as pd
from dataclasses import dataclass, asdict
from typing import List, Dict
from datetime import datetime

@dataclass
class ProcessoFinanceiro:
    """Representa um processo financeiro mapeado na organização.

    Attributes:
        nome: Nome descritivo do processo
        categoria: Classificação em operacional, analítico ou estratégico
        frequencia: Com que rotina o processo ocorre
        tempo_minutos: Tempo gasto por execução em minutos
        automatizavel: Se o processo pode ser automatizado com RPA/IA
        ferramenta_sugerida: Ferramenta recomendada para automação
    """
```

```

        responsavel_atual: Quem realiza o processo hoje
        custo_por_execuacao: Custo estimado por execução (em euros)
        erro_rate_pct: Taxa de erro histórico (em percentual)
    """
    nome: str
    categoria: str # operacional, analitico, estrategico
    frequencia: str # diaria, semanal, mensal, trimestral
    tempo_minutos: float
    automatizavel: bool
    ferramenta_sugerida: str
    responsavel_atual: str
    custo_por_execuacao: float
    erro_rate_pct: float

def mapear_processos_fornecedor() -> List[ProcessoFinanceiro]:
    """Mapeia processos financeiros típicos de fornecedor de odontologia.

    Retorna uma lista completa de processos, incluindo tempos,
    custos e taxas de er

```

Passo 2: Avaliar a Maturidade Digital

```

# avaliar_maturidade.py
# Framework PASSO - Passo 2: Avaliação de Maturidade Digital
# Avalia se a organização está pronta para automação RPA/IA
# em seus processos financeiros.

from enum import Enum
from dataclasses import dataclass
from typing import List, Dict

class NivelMaturidade(Enum):
    """Níveis de maturidade digital, do mais básico ao mais avançado."""
    INICIANTE = 1 # Processos 100% em papel
    BASICO = 2 # Algumas planilhas, mas sem integração
    INTERMEDIARIO = 3 # Sistemas digitais com exportação de dados
    AVANÇADO = 4 # Sistemas integrados com APIs
    OTIMIZADO = 5 # Automação parcial já implementada

@dataclass
class CriterioMaturidade:
    """Define um critério de avaliação de maturidade digital.

    Attributes:
        nome: Identificador do critério
        descricao: Descrição do que está sendo avaliado
        nivel_minimo: Nível mínimo necessário para automação
        indicadores: Lista de indicadores concretos para avaliação
        peso: Peso do critério no cálculo geral (1-5)
    """
    nome: str
    descricao: str
    nivel_minimo: NivelMaturidade
    indicadores: List[str]
    peso: int

```

```
# Definição dos critérios de maturidade
CRITERIOS = [
    CriterioMaturidade(
        nome="sistemas_em_uso",
        descricao="Sistemas de gestão e financeira implementados",
        nivel_minimo=NivelMaturidade.BASICO,
        indicadores=[
            "Software de gestão de fornecedores ou ERP",
            "Software financeiro para contas a pagar/receber",
            "Planilha de controle de estoque ou WMS básico"
        ]
    )
]
```

Passo 5: Otimizar com Dados Reais

```
# medir_impacto.py
# Framework PASSO - Passo 5: Medição de Impacto
# Calcula a economia real após implementação de automação,
# comparando tempos manuais com tempos automatizados.

from dataclasses import dataclass
from typing import List

@dataclass
class MetricaAutomacao:
    """Armazena os dados antes e depois da automação de um processo.

    Attributes:
        nome_processo: Nome do processo avaliado
        tempo_manual_min: Tempo gasto antes da automação (minutos)
        tempo_automatizado_min: Tempo gasto depois da automação (minutos)
        erros_manuais_mes: Erros encontrados por mês no processo manual
        erros_automatizados_mes: Erros encontrados por mês no processo automatizado
        custo_hora_analista: Custo hora do analista responsável (em euros)
    """
    nome_processo: str
    tempo_manual_min: float
    tempo_automatizado_min: float
    erros_manuais_mes: int
    erros_automatizados_mes: int
    custo_hora_analista: float

def calcular_antes_depois(metrica: MetricaAutomacao, dias_uteis_mes: int = 22) -> dict:
    """Calcula economia real de um processo após automação.

    Args:
        metrica: Dados do processo antes e depois da automação
        dias_uteis_mes: Número de dias úteis no mês (padrão: 22)

    Returns:
        Dicionário com todas as métricas de economia calculadas
    """
    economia_tempo_dia = metrica.tempo_manual_min - metrica.tempo_automatizado_min
    economia_tempo_mes = economia_tempo_dia * dias_uteis_mes
    economia_financeira_mes = (economia_tempo_mes / 60) * metrica.custo_hora_analista
```


if metrica.

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Faturas passaram a ser classificadas e conciliadas automaticamente (de 4 horas para 20 minutos)
- ☐ Relatórios financeiros eram gerados no dia 3 do fechamento (não no dia 12)
- ☐ O contador dedicou 3 horas por dia a análise de margem por procedimento
- ☐ Descobriu que materiais de restauração tinham margem 40% maior que materiais de limpeza
- ☐ A diretoria ajustou a política comercial com base nesse dado
- ☐ A Clínica Sorriso recebeu um relatório personalizado mostrando que, apesar do preço unitário maior, o fornecedor oferecia economia total de 8% quando considerava frete, prazo de entrega e condições de pagamento
- ☐ Automatizar sem mapear:** pular o Passo 1 (mapeamento) é o erro mais caro. Você pode acabar automatizando um processo que deveria ser eliminado

⑦ Armadilhas

- Automatizar sem mapear:** pular o Passo 1 (mapeamento) é o erro mais caro. Você pode acabar automatizando um processo que deveria ser eliminado
- Comprar ferramenta antes de avaliar maturidade:** se os dados estão em papel, nenhum RPA vai salvar você. Digitalize primeiro
- Ignorar a resistência da equipe:** a melhor automação do mundo falha se a equipe se recusa a usar. Comunicação clara é tão importante quanto o código
- Não medir depois de implementar:** “Parece que melhorou” não é métrica. Exija números antes e depois
- Automatizar processos estratégicos:** análise de margem e previsão de demanda são tarefas para humanos potencializados por IA, não para RPA puro

Passo 2 — Entendendo a Dinâmica de Compras do Dentista Português

Estágio: Passe • **Origem:** Cap. 2 — Entendendo a Dinâmica de Compras do Dentista Português

① Objetivo do passo

Ensinar o leitor a diferenciar equipamentos de capital de materiais de consumo e como cada categoria impacta o fluxo de caixa do fornecedor.

② Pré-requisito

Passo 1 concluído

③ Entregas

- (a completar)

④ Execução

Classificando Transações: Capital vs. Consumível

```
# classificar_transacoes.py
# Classifica transações de fornecedor de odontologia em
# capital ou consumível, calculando métricas de ciclo por categoria.
# Uso: python classificar_transacoes.py [--arquivo DADOS.csv]

import pandas as pd
from datetime import datetime, timedelta
from dataclasses import dataclass
from typing import List, Dict, Optional

# =====
# CONFIGURAÇÃO DE CLASSIFICAÇÃO
# Ajuste estes valores conforme o catálogo de produtos
# do fornecedor. Valores em euros.
# =====
LIMITE_CAPITAL = 500.00 # Itens acima deste valor = equipamento de capital
LIMITE_CICLO_LONGO = 45 # Dias entre compras do mesmo item para ser "longo"

@dataclass
class Transacao:
    """Representa uma transação de venda de produto odontológico.

    Attributes:
        produto: Nome do produto vendido
        valor: Preço unitário em euros
        data_compra: Data da transação (formato YYYY-MM-DD)
        fornecedor: Nome do fornecedor
        quantidade: Quantidade vendida (padrão: 1)
        cliente: Nome do cliente (clínica odontológica)
    """
    produto: str
    valor: float
    data_compra: str
    fornecedor: str
    quantidade: int = 1
    cliente: str = ""

def classificar_produto(valor_unitario: float) -> str:
    """Classifica um produto como capital ou consumível.

    Args:
        valor_unitario: Preço unitário do produto em euros
```

```

Returns:
    'capital' se valor >= LIMITE_CAPITAL, 'consumivel' caso contrário
    """
    return "capital" if va

```

Análise de Sazonalidade e Previsibilidade de Receita

```

# analisar_sazonalidade.py
# Analisa sazonalidade e previsibilidade de receita de
# fornecedor de odontologia usando média móvel e
# coeficiente de variação.
# Uso: python analisar_sazonalidade.py

import pandas as pd
import numpy as np
from datetime import datetime
from typing import Dict, Tuple

def analisar_sazonalidade(
    faturamentos_mensais: Dict[str, float],
    meses_previsao: int = 3
) -> Tuple[pd.DataFrame, Dict]:
    """Analisa sazonalidade e previsibilidade de receita.

    Args:
        faturamentos_mensais: Dicionário {mes_ano: valor_faturado}
            Ex: {"2024-01": 150000, "2024-02": 180000, ...}
        meses_previsao: Número de meses para média móvel (padrão: 3)

    Returns:
        Tupla (DataFrame com métricas mensais, dict com resumo executivo)
    """
    # Constrói DataFrame
    dados = [{"mes": k, "receita": v} for k, v in faturamentos_mensais.items()]
    df = pd.DataFrame(dados)
    df["mes"] = pd.to_datetime(df["mes"])
    df = df.sort_values("mes").reset_index(drop=True)

    # Média móvel de N meses (previsão de curto prazo)
    df["media_movel"] = df["receita"].rolling(window=meses_previsao).mean()

    # Coeficiente de variação (quanto a receita oscila)
    df["cv"] = df["receita"].rolling(window=meses_previsao).apply(
        lambda x: np.std(x) / np.mean(x) if np.mean(x) > 0 else 0
    )

    # Classificação de previsibilidade
    def classificar_previsibilidade(cv):
        if pd.isna(cv):
            return "indeterminado"
        if cv < 0.10:
            return "muito_alta"
        elif cv < 0.20:
            return "alta"

```

Dashboard Resumido: Margem por Categoria

```

# calcular_margem.py
# Calcula margem bruta por categoria de produto e gera
# dashboard executivo para diretoria.
# Uso: python calcular_margem.py

import pandas as pd
from dataclasses import dataclass
from typing import List
from datetime import datetime

@dataclass
class VendaProduto:
    """Representa a venda de um produto com dados de margem.

    Attributes:
        produto: Nome do produto
        categoria: 'capital' ou 'consumivel'
        valor_venda: Preço de venda unitário em euros
        custo: Custo de aquisição unitário em euros
        quantidade: Quantidade vendida no período
    """
    produto: str
    categoria: str
    valor_venda: float
    custo: float
    quantidade: int

def calcular_margem_por_categoria(vendas: List[VendaProduto]) -> pd.DataFrame:
    """Calcula margem bruta por categoria de produto.

    Args:
        vendas: Lista de VendaProduto

    Returns:
        DataFrame com margem por categoria e ranking de rentabilidade
    """
    df = pd.DataFrame([vars(v) for v in vendas])
    df["receita"] = df["valor_venda"] * df["quantidade"]
    df["custo_total"] = df["custo"] * df["quantidade"]
    df["lucro_bruto"] = df["receita"] - df["custo_total"]
    df["margem_pct"] = (df["lucro_bruto"] / df["receita"] * 100).round(1)

    # Resumo por categoria
    resumo_cat = df.groupby("categoria").agg(
        receita_total=("receita", "sum"),
        custo_total=("custo_total", "sum"),
        lucro_total=("lucro_bruto", "sum"),
        num_skus=("produto", "count"),
        qtd_total=("quantidade", "sum")
    ).round(2)

    resumo_cat[

```

Construindo o Dashboard Integrado

```

# dashboard_integrado.py
# Integra classificação, sazonalidade e margem em um
# painel executivo único para diretoria.
# Uso: python dashboard_integrado.py

import pandas as pd
from datetime import datetime
from typing import Dict

def calcular_dias_de_caixa(
    caixa_atual: float,
    despesas_fixas_mensais: float,
    despesas_variaveis_mensais: float,
    receita_media_diaria: float
) -> Dict:
    """Calcula indicadores de saúde de caixa.

    Args:
        caixa_atual: Saldo atual em euros
        despesas_fixas_mensais: Despesas fixas (aluguel, salários, etc.)
        despesas_variaveis_mensais: Despesas variáveis (estoque, comissões)
        receita_media_diaria: Receita média diária dos últimos 30 dias

    Returns:
        Dicionário com indicadores de caixa
    """
    despesas_diarias = (despesas_fixas_mensais + despesas_variaveis_mensais) / 22
    dias_de_caixa = caixa_atual / despesas_diarias if despesas_diarias > 0 else
float("inf")
    caixa_projetado_30 = caixa_atual + (receita_media_diaria * 30) - (despesas_diarias *
30)

    if dias_de_caixa < 15:
        classificacao = "CRÍTICO"
    elif dias_de_caixa < 30:
        classificacao = "ATENÇÃO"
    elif dias_de_caixa < 60:
        classificacao = "SAUDÁVEL"
    else:
        classificacao = "FORTALECIDO"

    return {
        "dias_de_caixa": round(dias_de_caixa, 1),
        "classificacao": classificacao,
        "caixa_projetado_30_dias": round(caixa_projetado_30, 2),
        "despesas_diarias": round(despesas_diarias, 2),
        "gap_receita_despesa": round(receita_media_diaria - despesas_diaria

```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ❑ Misturar caixas:** usar o caixa de consumíveis para cobrir furos de equipamentos é o caminho mais rápido para uma crise de liquidez. São dois rios — mantenha-os separados

- ☐ Ignorar sazonalidade:** planejar fluxo de caixa sem considerar os picos de janeiro e setembro é como dirigir sem olhar o GPS — você vai se perder, e a surpresa vai custar caro
- ☐ Não classificar transações:** quando tudo é “compra”, o analista perde a visibilidade de onde está o risco e onde está a âncora. Sem classificação, não há estratégia
- ☐ Esquecer o ciclo de pagamento:** equipamentos frequentemente têm pagamento parcelado em 3 a 6 parcelas — o impacto no caixa se espalha, não acontece de uma vez. Proje que €12.000 em parcelas de €2.000 é muito diferente de €12.000 à vista
- ☐ Não medir previsibilidade:** se você não sabe quanto da receita do próximo mês pode projetar com confiança, está dirigindo no escuro. O coeficiente de variação é seu GPS

⑦ Armadilhas

- Misturar caixas:** usar o caixa de consumíveis para cobrir furos de equipamentos é o caminho mais rápido para uma crise de liquidez. São dois rios — mantenha-os separados
- Ignorar sazonalidade:** planejar fluxo de caixa sem considerar os picos de janeiro e setembro é como dirigir sem olhar o GPS — você vai se perder, e a surpresa vai custar caro
- Não classificar transações:** quando tudo é “compra”, o analista perde a visibilidade de onde está o risco e onde está a âncora. Sem classificação, não há estratégia
- Esquecer o ciclo de pagamento:** equipamentos frequentemente têm pagamento parcelado em 3 a 6 parcelas — o impacto no caixa se espalha, não acontece de uma vez. Proje que €12.000 em parcelas de €2.000 é muito diferente de €12.000 à vista
- Não medir previsibilidade:** se você não sabe quanto da receita do próximo mês pode projetar com confiança, está dirigindo no escuro. O coeficiente de variação é seu GPS

Passo 3 — Criando a Ponte Comercial-Financeiro

Estágio: Acesso · **Origem:** Cap. 3 — Criando a Ponte Comercial-Financeiro

① Objetivo do passo

Construir uma matriz que cruza dados do funil de vendas com previsões de faturamento, eliminando o silo entre departamentos.

② Pré-requisito

Passo 2 concluído

③ Entregas

- (a completar)

④ Execução

Código Completo: Motor de Pipeline Ponderado

```

"""
Motor de Pipeline Ponderado – Capítulo 3
Conecta dados comerciais (funil) a previsões financeiras (faturamento)
"""

from dataclasses import dataclass, field
from typing import Optional
import json
from datetime import datetime, timedelta

@dataclass
class Deal:
    """Representa um deal/comercial no funil de vendas."""
    id_deal: str
    cliente: str
    categoria: str # "equipamento" ou "consumível"
    valor_bruto: float
    estagio: str
    data_abertura: datetime
    vendedor: str
    regioao: str = "SP"

@dataclass
class EstagioFunil:
    """Define um estágio do funil com suas propriedades."""
    nome: str
    ordem: int
    probabilidade_padrao: float # 0.0 a 1.0
    dias_medios: int # tempo médio neste estágio

# Configuração do funil padrão para distribuidoras odontológicas B2B
ESTAGIOS_PADRAO = [
    EstagioFunil("Prospeccao", 1, 0.10, 30),
    EstagioFunil("Qualificacao", 2, 0.25, 25),
    EstagioFunil("Proposta Enviada", 3, 0.50, 20),
    EstagioFunil("Negociacao", 4, 0.75, 15),
    EstagioFunil("Fechamento", 5, 0.90, 10),
]

def carregar_deals_do_caminho(caminho_json: str) -> list[Deal]:
    """Carrega deals a partir de um arquivo JSON exportado do CRM."""
    with open(caminho_json, "r", encoding="utf-8") as f:
        dados = json.load(f)
    deals = []
    for registro in dados:
        deals.append(
            Deal(
                id_deal=registro["id"],
                cliente=registro["cliente"],
                categoria=registro["categoria"],
                valor_bruto=registro["valor"],
                estagio=registro["estagio"],

```

```
// =====
// MEDIDAS DO PIPELINE PONDERADO – Power BI DAX
// Conecta dados do CRM (comercial) ao ERP (financeiro)
// =====

// 1. Pipeline Bruto Total
Pipeline Bruto =
SUM ( 'CRM_Deals'[Valor_Bruto] )

// 2. Pipeline Ponderado (com probabilidade por estágio)
Pipeline Ponderado =
SUMX (
    'CRM_Deals',
    'CRM_Deals'[Valor_Bruto] *
    RELATED ( 'Dim_Estagios'[Probabilidade] )
)

// 3. Conversão Esperada (%)
Conversao Esperada =
DIVIDE (
    [Pipeline Ponderado],
    [Pipeline Bruto],
    0
)

// 4. Previsão de Faturamento (próximos 90 dias)
Previsao Faturamento 90d =
CALCULATE (
    SUM ( 'ERP_Faturamento'[Valor_Liquido] ),
    FILTER (
        ALL ( 'ERP_Faturamento'[Data_Emissao] ),
        'ERP_Faturamento'[Data_Emissao] >= TODAY ()
        && 'ERP_Faturamento'[Data_Emissao] <= TODAY () + 90
    )
)

// 5. Previsibilidade (erro médio absoluto das últimas 12 previsões)
Previsibilidade =
VAR ErrosAbsolutos =
    ADDCOLUMNS (
        VALUES ( 'Historico_Previsoes'[Mes] ),
        "Erro",
        ABS (
            CALCULATE ( [Previsao Faturamento 90d] )
            - CALCULATE ( [Pipeline Ponderado] )
        )
    )
RETURN
    1 - ( AVERAGEX ( ErrosAbsolutos, [Erro] )
        / [Pipeline Ponderado] )

// 6. Margem Bruta por Categoria
Margem por Categoria =
SUMX (
    'CRM_Deals',
    'CRM_Deals'[Valor_Bruto]
    * RELATED ( 'Dim_Categorias'[Margem_Bruta] )
    * RELATED ( 'Dim_Estagios'[Probabilidade] )
)
```



```
)

// 7. Dias Médios no Funil
Dias Medios F
```

Script de Automação: Sincronização CRM → ERP

```
"""
Sincronizador CRM → ERP – Capítulo 3
Automatiza a ponte comercial-financeiro com atualização diária.
"""

import requests
import csv
from datetime import datetime, date
from pathlib import Path

class SincronizadorPontBridge:
    """Sincroniza dados do CRM para o ERP, calculando pipeline ponderado."""

    CRM_API_URL = "https://api.crm-empresa.com/v1/deals"
    ERP_IMPORT_DIR = Path("./imports/erp/")
    LOG_DIR = Path("./logs/sincronizacao/")

    def __init__(self, api_key: str, empresa_id: str):
        self.headers = {
            "Authorization": f"Bearer {api_key}",
            "Content-Type": "application/json",
        }
        self.empresa_id = empresa_id
        self.LOG_DIR.mkdir(parents=True, exist_ok=True)
        self.ERP_IMPORT_DIR.mkdir(parents=True, exist_ok=True)

    def buscar_deals_ativos(self) -> list[dict]:
        """Busca todos os deals ativos do CRM via API."""
        params = {
            "empresa_id": self.empresa_id,
            "status": "ativo",
            "campos": "id,cliente,categoria,valor,estagio,data_abertura,vendedor",
        }
        resposta = requests.get(
            self.CRM_API_URL, headers=self.headers, params=params, timeout=30
        )
        resposta.raise_for_status()
        return resposta.json()["deals"]

    def calcular_ponderacao(self, deals: list[dict]) -> list[dict]:
        """Aplica probabilidades de conversão a cada deal."""
        probabilidades = {
            "prospeccao": 0.10,
            "qualificacao": 0.25,
            "proposta_enviada": 0.50,
            "negociacao": 0.75,
            "fec
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Confundir pipeline bruto com previsão.** O erro mais custoso. R\ \$ 4 milhões em pipeline não são R\ \$ 4 milhões em receita. Sem a ponderação por probabilidade, você está projetando um cenário que provavelmente não vai acontecer
- ☐ Ignorar a segmentação por categoria.** Consumíveis e equipamentos têm dinâmicas de conversão completamente diferentes. Tratá-los iguais na matriz produz uma previsão medíocre para ambos
- ☐ Não validar com o comercial.** A probabilidade que o financeiro calcula nos dados históricos pode não refletir a realidade atual. O vendedor que está negociando o deal sabe coisas que os números não capturam — use essa informação
- ☐ Sincronizar demais ou de menos.** Atualizar a matriz uma vez por mês é insuficiente no B2B odontológico, onde decisões de compra mudam rápido. Mas atualizar todos os dias gera ruído. O ritmo ideal é semanal
- ☐ Apresentar dados sem recomendação.** Dados sem direção são ruído. A diretoria não quer saber “o que aconteceu” — quer saber “o que fazer com isso”

⑦ Armadilhas

- (a completar)

Passo 4 — O Seu Plano de Ação Imediato

Estágio: Acesso · **Origem:** Cap. 4 — O Seu Plano de Ação Imediato

① Objetivo do passo

Entregar ao leitor um template de apresentação enxuta para mostrar seus primeiros achados estratégicos à diretoria no final do mês.

② Pré-requisito

Passo 3 concluído

③ Entregas

- (a completar)

④ Execução

Template Python Completo: Gerador de Relatório Executivo

```
"""
Gerador de Relatório Executivo – Capítulo 4
Transforma dados da matriz comercial-financeiro em apresentação de 5 slides.
```

```
"""

from dataclasses import dataclass
from datetime import datetime
from typing import Optional
import json

@dataclass
class AchadoEstrategico:
    """Representa um achado selecionado para o Slide 3."""
    titulo: str
    categoria: str
    valor_atual: float
    valor_anterior: float
    variacao_pct: float
    impacto_financeiro: float
    unidade: str = "R\\$"

@dataclass
class Recomendacao:
    """Representa uma recomendação para o Slide 4."""
    acao: str
    responsavel: str
    prazo: str
    impacto: str
    metrica_sucesso: str

@dataclass
class ProximoPasso:
    """Representa um próximo passo para o Slide 5."""
    acao: str
    responsavel: str
    prazo: str
    dependencia: Optional[str] = None

class GeradorRelatorioExecutivo:
    """Gera relatório executivo de 5 slides para diretoria."""

    def __init__(
        self,
        empresa: str,
        trimestre: str,
        dados_pipeline: dict,
    ):
        self.empresa = empresa
        self.trimestre = trimestre
        self.dados_pipeline = dados_pipeline

    def selecionar_achados(
        self, top: int = 3
    ) -> list[AchadoEstrategico]:
        """
        Seleciona os 3 achados com maior variação percentual
        entre categorias de produto.
        """
```

```

"""
categorias = self.dados_pipeline.get("por_categoria", {})
achados = []

for cat, dados in categorias.items():
    bruto = dados.get("bruto", 0)
    ponderado = dados.get("ponde

```

Script de Automação: Gerador de Slides Markdown

```

"""
Gerador de Slides Markdown – Capítulo 4
Conecta ao ERP e gera apresentação executiva automaticamente.
"""

import sqlite3
from datetime import datetime, timedelta
from pathlib import Path

class GeradorSlidesMarkdown:
    """Gera slides Markdown a partir dos dados do ERP."""

    def __init__(self, caminho_banco: str):
        self.conn = sqlite3.connect(caminho_banco)
        self.conn.row_factory = sqlite3.Row
        self.output_dir = Path("./output/apresentacoes/")
        self.output_dir.mkdir(parents=True, exist_ok=True)

    def buscar_pipeline_por_categoria(self) -> list[dict]:
        """Busca o pipeline agrupado por categoria de produto."""
        query = """
        SELECT
            c.nome as categoria,
            COUNT(d.id) as qtd_deals,
            SUM(d.valor) as valor_bruto,
            SUM(d.valor * e.probabilidade) as valor_ponderado,
            AVG(d.dias_no_estagio) as dias_medios
        FROM deals d
        JOIN categorias c ON d.categoria_id = c.id
        JOIN estagios e ON d.estagio_id = e.id
        WHERE d.status = 'ativo'
        GROUP BY c.nome
        ORDER BY valor_ponderado DESC
        """
        cursor = self.conn.execute(query)
        return [dict(row) for row in cursor.fetchall()]

    def buscar_top_clientes(
        self, limite: int = 50
    ) -> list[dict]:
        """Busca os maiores clientes por valor de pipeline."""
        query = """
        SELECT
            cl.nome,
            cl.tipo,
            COUNT(d.id) as qtd_deals,

```

```
SUM(d.valor) as valor_total,
SUM(d.valor * e.probabilidade) as val
```

Checklist Automatizado: Validação Pré-Apresentação

```
"""
Validador de Apresentação Executiva – Capítulo 4
Verifica se a apresentação atende aos critérios de qualidade.
"""

from dataclasses import dataclass

@dataclass
class ResultadoValidacao:
    """Resultado de uma verificação do checklist."""
    item: str
    status: bool # True = aprovado, False = problema
    detalhe: str

def validar_apresentacao(
    achados: list[dict],
    recomendacao: str,
    proximos_passos: list[dict],
    tempo_total_min: float,
) -> list[ResultadoValidacao]:
    """
    Valida a apresentação executiva contra os critérios de qualidade.
    Retorna lista de resultados com status de cada verificação.
    """

    resultados = []

    # 1. Máximo 3 achados
    qtd_achados = len(achados)
    resultados.append(
        ResultadoValidacao(
            item="Máximo 3 achados",
            status=qtd_achados <= 3,
            detalhe=(
                f"{qtd_achados} achados selecionados "
                f"(máximo: 3)"
            ),
        )
    )

    # 2. Recomendação tem número específico
    tem_numero = any(
        c.isdigit() or "%" in recomendacao
        for c in recomendacao
    )
    resultados.append(
        ResultadoValidacao(
            item="Recomendação com número específico",
            status=tem_numero,
            detalhe=(
                "Recomendação contém quantificador"
                if tem_numero
            )
        )
    )
```

```

        else "Recomendação genérica – adicionar números"
    ),
)
)

# 3. Recomendação tem prazo
tem_prazo = any(
    palavra in recomendacao.lower()
    for palavra in ["até", "

```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Sobrecarga informacional.** Mais de 3 achados por apresentação dilui o impacto. O executivo não consegue reter mais do que isso em uma reunião. Se você tem 10 achados, selecione os 3 com maior impacto financeiro e guarde o resto para o relatório completo
- ☐ Dados sem contexto.** Um número sozinho não significa nada. “Receita caiu 12%” é diferente de “Receita caiu 12% enquanto o mercado cresceu 8%.” Sem o comparativo, o dado é ruído — e ruído não gera decisão
- ☐ Recomendação vaga.** “Melhorar a eficiência” não é uma recomendação. É um desejo. A diretoria precisa de ação: quem, o quê, quando. Se sua recomendação não cabe em uma frase com responsável e prazo, ela não está pronta para a sala de decisões
- ☐ Esquecer o Slide 5.** Muitos analistas param na recomendação. Mas sem os próximos passos com responsável e prazo, a reunião termina com “vamos pensar nisso” — que é o epitáfio de milhares de boas ideias. O Slide 5 é o que transforma uma apresentação em uma decisão executável
- ☐ Não praticar o tempo.** 10 minutos é pouco. Se você não ensaiou, vai passar do tempo — e o CEO vai cortar sua fala antes de você chegar na recomendação. Pratique com cronômetro. Cada slide deve caber exatamente no tempo designado

⑦ Armadilhas

- (a completar)

Checklist Mestre

Passo 1 — O Fim do Digitador de Faturas

- ☐ Faturas passaram a ser classificadas e conciliadas automaticamente (de 4 horas para 20 minutos)
- ☐ Relatórios financeiros eram gerados no dia 3 do fechamento (não no dia 12)
- ☐ O contador dedicou 3 horas por dia a análise de margem por procedimento
- ☐ Descobriu que materiais de restauração tinham margem 40% maior que materiais de limpeza
- ☐ A diretoria ajustou a política comercial com base nesse dado
- ☐ A Clínica Sorriso recebeu um relatório personalizado mostrando que, apesar do preço unitário maior, o fornecedor oferecia economia total de 8% quando considerava frete, prazo de entrega e condições de pagamento
- ☐ Automatizar sem mapear:** pular o Passo 1 (mapeamento) é o erro mais caro. Você pode acabar automatizando um processo que deveria ser eliminado

Passo 2 — Entendendo a Dinâmica de Compras do Dentista Português

- ☐ Misturar caixas:** usar o caixa de consumíveis para cobrir furos de equipamentos é o caminho mais rápido para uma crise de liquidez. São dois rios — mantenha-os separados
- ☐ Ignorar sazonalidade:** planejar fluxo de caixa sem considerar os picos de janeiro e setembro é como dirigir sem olhar o GPS — você vai se perder, e a surpresa vai custar caro
- ☐ Não classificar transações:** quando tudo é “compra”, o analista perde a visibilidade de onde está o risco e onde está a âncora. Sem classificação, não há estratégia
- ☐ Esquecer o ciclo de pagamento:** equipamentos frequentemente têm pagamento parcelado em 3 a 6 parcelas — o impacto no caixa se espalha, não acontece de uma vez. Proje que €12.000 em parcelas de €2.000 é muito diferente de €12.000 à vista
- ☐ Não medir previsibilidade:** se você não sabe quanto da receita do próximo mês pode projetar com confiança, está dirigindo no escuro. O coeficiente de variação é seu GPS

Passo 3 — Criando a Ponte Comercial-Financeiro

- ☐ Confundir pipeline bruto com previsão.** O erro mais custoso. R\$ 4 milhões em pipeline não são R\$ 4 milhões em receita. Sem a ponderação por probabilidade, você está projetando um cenário que provavelmente não vai acontecer
- ☐ Ignorar a segmentação por categoria.** Consumíveis e equipamentos têm dinâmicas de conversão completamente diferentes. Tratá-los iguais na matriz produz uma previsão medíocre para ambos
- ☐ Não validar com o comercial.** A probabilidade que o financeiro calcula nos dados históricos pode não refletir a realidade atual. O vendedor que está negociando o deal sabe coisas que os números não capturam — use essa informação
- ☐ Sincronizar demais ou de menos.** Atualizar a matriz uma vez por mês é insuficiente no B2B odontológico, onde decisões de compra mudam rápido. Mas atualizar todos os dias gera ruído. O ritmo ideal é semanal
- ☐ Apresentar dados sem recomendação.** Dados sem direção são ruído. A diretoria não quer saber “o que aconteceu” — quer saber “o que fazer com isso”

Passo 4 — O Seu Plano de Ação Imediato

- ☐ Sobrecarga informacional.** Mais de 3 achados por apresentação dilui o impacto. O executivo não consegue reter mais do que isso em uma reunião. Se você tem 10 achados, selecione os 3 com maior impacto financeiro e guarde o resto para o relatório completo
- ☐ Dados sem contexto.** Um número sozinho não significa nada. “Receita caiu 12%” é diferente de “Receita caiu 12% enquanto o mercado cresceu 8%.” Sem o comparativo, o dado é ruído — e ruído não gera decisão
- ☐ Recomendação vaga.** “Melhorar a eficiência” não é uma recomendação. É um desejo. A diretoria precisa de ação: quem, o quê, quando. Se sua recomendação não cabe em uma frase com responsável e prazo, ela não está pronta para a sala de decisões
- ☐ Esquecer o Slide 5.** Muitos analistas param na recomendação. Mas sem os próximos passos com responsável e prazo, a reunião termina com “vamos pensar nisso” — que é o epitáfio de milhares de boas ideias. O Slide 5 é o que transforma uma apresentação em uma decisão executável
- ☐ Não praticar o tempo.** 10 minutos é pouco. Se você não ensaiou, vai passar do tempo — e o CEO vai cortar sua fala antes de você chegar na recomendação. Pratique com cronômetro. Cada slide deve caber exatamente no tempo designado